

TraceGraph – Technical Documentation

Architecture and Code Explanation for Phase 3

1. System Purpose

TraceGraph is a graph-based retrieval and question answering system designed to support multi-hop reasoning over long text collections. Traditional top- k retrieval often returns isolated chunks that are individually relevant but insufficient to answer questions requiring multiple connected pieces of evidence. The goal of TraceGraph is to address this limitation by representing document chunks as graph nodes and using both retrieval and graph traversal to identify connected evidence.

The system is designed as a modular pipeline. It prepares source documents, converts them into chunks, builds graph relationships among those chunks, retrieves strong seed evidence, traverses related nodes, assembles context, and finally generates answers with citations and path explanations. This design makes the prototype suitable for experimentation, evaluation, and future extension.

2. High-Level Architecture

The repository is organized into several major modules, each with a distinct responsibility in the pipeline.

Module	Role in the System
data/	Handles corpus loading, dataset preparation, chunking, preprocessing, and metadata enrichment.
graph/	Builds the chunk graph, defines edge-creation logic, and saves graph structures to disk.
indexing/	Constructs retrieval indices such as BM25, entity matching, and optional embeddings.
retrieval/	Performs seed retrieval, graph traversal, context assembly, citation formatting, and retrieval-path explanation.
agent/	Controls answer drafting, missing-evidence detection, query refinement, and callback-based reasoning loops.
evaluation/	Computes metrics such as Exact Match and F1, runs ablations, and writes evaluation summaries.
config/ and scripts/	Store YAML configuration files and runnable command-line entry points.

This structure separates responsibilities cleanly, which improves maintainability and makes it easier to test individual parts of the system independently.

3. Core Processing Flow

The main program flow starts from the command-line interface, which acts as the entry point for multiple workflows such as dataset preparation, graph building, retrieval, answering, evaluation, ablations, and

reporting.

The overall technical sequence is as follows:

1. Load input documents or a prepared evaluation dataset.
2. Split documents into chunks using the chunking pipeline.
3. Enrich chunks with metadata such as entities and keywords.
4. Insert chunks into a graph as nodes.
5. Add edges between nodes based on structural or semantic relationships.
6. Build retrieval indices for the chunks.
7. Retrieve top seed nodes for a given query.
8. Traverse neighboring nodes in the graph to collect connected evidence.
9. Assemble the retrieved evidence into a compact context.
10. Use the answering agent to produce a final answer with supporting citations.
11. Optionally run evaluation and write reports.

This flow ensures that retrieval is not limited to isolated chunks. Instead, the system can revisit related evidence and provide a more complete context for answer generation.

4. Data Preparation Layer

The data preparation layer is responsible for converting raw text into graph-ready units. The input corpus is first loaded from its source representation. The text is then segmented into chunks of manageable size, which helps both retrieval and graph construction. These chunks are small enough to be scored efficiently, but large enough to preserve useful context.

After chunking, the preprocessing logic enriches each chunk by extracting keywords, entities, or other metadata. These enriched chunk objects become the foundation for later graph edges and retrieval features. The result of this stage is a structured set of chunk records containing identifiers, source references, text content, and derived metadata.

5. Graph Construction

The graph layer converts chunks into a sparse multi-edge graph. Each chunk is represented as a node, and edges are created based on different types of relationships. This allows the system to represent not just textual relevance, but also how pieces of evidence connect to one another.

The graph currently includes several major edge types:

- **NEXT**: connects adjacent chunks from the same document to preserve sequential order.
- **SHARED_ENTITY**: links chunks that mention common named entities or extracted concepts.
- **LEXICAL_SIM**: connects chunks with strong lexical or semantic similarity.
- **RELATION**: optionally connects chunks when higher-level associations are detected.

The graph builder validates node identities, inserts nodes, creates eligible edges, and computes graph statistics. A multi-edge design is useful here because the same two chunks may be connected for multiple reasons, such as shared entities and lexical similarity at the same time.

6. Indexing and Seed Retrieval

TraceGraph does not rely on a single retrieval signal. Instead, it builds multiple retrieval views over the same chunk collection. This improves robustness because different retrieval methods capture different aspects of relevance.

The indexing layer supports:

- BM25 for lexical matching,
- entity overlap for concept-based retrieval,
- optional embedding similarity for semantic retrieval.

During query processing, the seed retriever gathers candidate chunks from the enabled indices. Since the score ranges from each index are not naturally comparable, the system normalizes them before combining them. Weighted score fusion is then used to compute a final ranking. This means that chunks are selected not simply because they are highly ranked by one method, but because they remain strong across multiple retrieval perspectives.

This retrieval strategy is one of the central algorithmic contributions in the prototype because it provides a more stable starting point for graph traversal.

7. Graph Traversal and Context Assembly

After seed retrieval, the system optionally expands outward through the graph to gather connected evidence. This is handled by the traversal module, which explores neighboring nodes while respecting configured limits such as traversal depth, expansion budget, and token budget.

Traversal is especially useful for multi-hop questions. In many cases, the seed chunks identify one relevant concept but do not contain the complete answer. By moving through graph edges, the system can discover bridge evidence that links related ideas across different chunks or documents.

The output of traversal is then passed to the context assembly stage. At this point, the retrieved nodes are ordered, filtered, and merged into a compact textual context. Citation formatting and retrieval-path explanation are also produced so that the final answer can indicate where the supporting evidence came from.

8. Agent Layer and Answer Generation

The answering component is controlled through the agent layer. Rather than treating answer generation as a single fixed operation, the system allows both single-pass answering and a revisitable callback loop.

In the single-pass mode, the agent receives the assembled context and generates an answer directly. In the callback mode, the agent drafts an initial answer, checks whether supporting evidence appears incomplete, and can refine the query for another round of retrieval. This loop continues until sufficient evidence is found or a configured callback limit is reached.

This design is important because it mirrors how difficult questions are often solved in practice. A first retrieval attempt may surface only partial evidence, so the system benefits from being able to revisit retrieval with a refined objective.

9. Important Code Modules

Several implementation files play especially important roles in the current repository:

File / Module	Purpose
graph/builder.py	Creates the graph from chunks, adds nodes and edges, validates identifiers, and computes graph statistics.
indexing/seed_retriever.py	Retrieves candidates from multiple indices, normalizes scores, fuses them, and ranks top seed nodes.
retrieval/retriever.py	Coordinates seed retrieval, graph traversal, context assembly, and retrieval explanation.
retrieval/traversal.py	Expands through graph edges under configurable depth and budget constraints.
agent/callbacks.py	Implements the retrieve-draft-detect-refine loop for iterative evidence gathering.
agent/tracegraph_agent.py	Serves as the main interface for running answer generation with or without callbacks.
evaluation/metrics.py	Computes normalized Exact Match and token-level F1 metrics.
data/models.py	Defines structured models such as documents, chunks, retrieved nodes, retrieval bundles, and answer outputs.

Together, these modules form the backbone of the system.

10. Configuration and Output Artifacts

The repository uses YAML-based configuration files to control major system behavior. These settings cover chunk size, graph edge rules, enabled retrieval modes, traversal depth, callback limits, evaluation datasets, and output paths. This makes the prototype easier to experiment with because behavior can be changed without rewriting core logic.

When the system runs, it produces several output artifacts, including:

- serialized graphs,
- processed chunk files,
- retrieval bundles,
- agent traces,
- evaluation summaries,
- generated reports.

These outputs help with debugging, reproducibility, and analysis during development.

11. Testing and Reliability

A useful strength of the current implementation is that it is not only modular but also testable. The repository includes deterministic tests for major components such as chunking, graph construction, traversal, metrics, citation behavior, persistence, and end-to-end demo workflows.

Testing is particularly important in this project because multiple modules interact closely. Errors in chunking can affect graph quality, and issues in graph quality can affect retrieval and answer generation. By validating each step independently, the system becomes more reliable and easier to extend.

12. Current Limitations

Although the current prototype is functional, several limitations remain. First, relation-based edges are still lighter-weight than a fully learned relational representation, so some deeper semantic links may be missed. Second, evidence grounding is primarily at the chunk level, which means sentence-level attribution can still be improved. Third, answer synthesis can be further strengthened through tighter evidence selection and stronger language-model integration.

These limitations do not prevent the system from functioning as a proof of concept, but they identify clear directions for the final phase.

13. Conclusion

The TraceGraph codebase implements a complete graph-based retrieval and answering pipeline suitable for Phase 3 prototype development. Its architecture covers data preparation, graph building, multi-source retrieval, graph traversal, answer generation, and evaluation. The code is modular, configurable, and supported by testing, which makes it suitable both as a working MVP-level prototype and as a strong foundation for future improvements.